CA Project List
[CO4]

Sr. No	Project Title	Type/Simulator	Contact TA
1.	Sparse-Aware RISC-V Vector Architecture for High performance Computing	Verilator or questaSim, RISC-V GNU toolchain, Spike	Sonam Email: sonami@iiitd.ac.in
2.	The MoE Memory Wall: Predicting and Prefetching Experts for Faster LLM Inference	Software/ LLM-Simulator	Ayesha Parveen Email: ayeshap@iiitd.ac.in
3.	Characterizing Memory Access Predictability Using Hardware Prefetcher Classes and improving the prefetcher.	Software/Champsim	Naorem Akshaykumar Email: naorema@iiitd.ac.in
4.	RTL Design & Verification of MESI Cache Coherence Protocol with Next-Line Prefetching	Hardware, any RTL simulator,	Shrishail Dolle & Naorem Akshaykumar Email : Shrishail Siddharam …
5.	Performance Characterization and Optimization of MoE Communication on Chiplet Based Architectures.	RapidChiplet Booksim2.0	Sameena Khan sameenak@iiitd.ac.in

General Guideline:

1. Pick any one of the projects listed above, and details are provided on the next page.
2. Use of AI tools such as ChatGPT, Gemini, and DeepSeek in **project report writing/PPT** is prohibited. Please note it's only about **project reports/PPTs**.
3. All the projects listed above take time to implement, so plan accordingly. For clarification of doubts, meet the TA only during their office hours. No TA will respond to any email sent during an unreasonable time or just before the due date.
4. The group will consist of at most 3 members. Each individual is expected to contribute equally to the project, and there will be an individual viva
5. Artifacts for the project and PPT need to be submitted at the final evaluation.

Plagiarism Policy

1. The project is to evaluate your own implementation of the project goals.
2. Your code WILL be cross-checked against any online code or materials, as well as submissions from other courses. Any reasonable similarity found will be subjected to the Academic Dishonesty Policy.

Project Rubrics:

Component	Marks	What is evaluated
Project Deliverables(Report+Demo)	10	Whether the stated project deliverable has been achieved and clearly documented in the final report and demonstrated through a working demo.
Experimental Analysis and Evaluation	4	Correct methodology, interpretation of results, discussion of limitation/failure case and demonstration of the experimental finding.
Artifact reproducibility	3	Complete code /configuration/scripts, README and dependencies, clear run commands, ability to reproduce at least one representative result and working demo of the artifact.
Presentation (PPT)	3	Clear and well organized presentation, readable figures/tables, proper explanation of the problem, methods, results and conclusion.
Individual Viva	5	Individual understanding of the project, implementation methodology, results and ability to explain the demo.
Total	25	

Project- 1 (Research paper opportunity)

Sparse-Aware RISC-V Vector Architecture for High performance Computing

Project Description

This is a RISC-V vector-architecture-based project. In this project, you are going to study and develop a **sparse-aware vector processing architecture** for accelerating sparse workloads on a RISC-V processor supporting the RISC-V Vector Extension (RVV).

Conventional vector processors are highly efficient for dense and regular workloads, but their efficiency can decrease significantly for sparse applications because many vector elements may be zero or inactive. Processing these elements can result in unnecessary computation, vector-register usage, and memory traffic.

In this project, you will first study the baseline RISC-V vector architecture and characterize its performance on sparse workloads. You will then investigate an architecture that can exploit sparsity during vector execution. The proposed design may include sparse-data detection, sparse metadata management, efficient vector memory access, zero/inactive-element skipping, and sparse reduction mechanisms. The primary application considered in this project will be **Sparse Matrix-Vector Multiplication (SpMV)**, which is widely used in high-performance computing and scientific applications. Additional workloads such as sparse matrix multiplication or graph-processing kernels may also be considered.

The project will investigate the trade-off between conventional dense vector execution and sparse vector execution. In particular, you will analyze the sparsity level at which sparse execution becomes beneficial and study the associated performance, memory-bandwidth, and hardware overheads.

Mid-evaluation Deliverable

1. Study and summarize the RISC-V Vector Extension and existing sparse-computing architectures.
2. Set up and validate the baseline RISC-V vector simulation environment.
3. Implement/run a baseline RVV processor with **SpMV** or an equivalent sparse workload.
4. Run the selected workload with at least **two different sparsity levels**.
5. Report the execution time and performance of the baseline RVV implementation.

End-evaluation Deliverable

1. Implement/model the sparse-aware RISC-V vector architecture.
2. The proposed design should include at least one hardware/software mechanism for exploiting sparsity, such as: zero/inactive-element skipping, sparse metadata support, sparse-aware vector execution, or sparse-aware memory access.
3. Run at least two sparse workloads, one of which should be the workload used during the mid-semester evaluation.
4. Evaluate the proposed architecture using multiple sparsity levels.

5. Report: Execution time, Speedup Vector utilization, Cache miss rate, Memory traffic/bandwidth, Number of useful versus skipped operations
6. Compare the proposed sparse-aware architecture with the baseline RVV implementation.
7. Analyze the performance benefit versus sparsity level and identify the point at which sparse execution becomes more beneficial than conventional dense execution.
8. Report the hardware overhead of the proposed mechanism, if an RTL implementation is performed.
9. Provide a final report describing the architecture, implementation methodology, experimental results, and conclusions.

References:

[1] RISC-V International, *RISC-V "V" Vector Extension*, RISC-V International, official specification.

[2] T. A. Davis et al., "The University of Florida Sparse Matrix Collection," *ACM Transactions on Mathematical Software*, vol. 38, no. 1, 2011.

[3] N. Bell and M. Garland, "Implementing Sparse Matrix-Vector Multiplication on Throughput-Oriented Processors," in *Proceedings of SC*, 2009.

[4] A. H. Sameh and K. K. K. "Sparse Matrix Computations," relevant literature on sparse linear algebra and HPC architectures.

[5] Relevant RISC-V documentation for the selected simulation environment.

TA contact:

Sonam
Email: sonami@iiitd.ac.in

Project 2: The MoE Memory Wall: Predicting and Prefetching Experts for Faster LLM Inference

Description:

Mixture-of-Experts (MoE) models activate only a small subset of experts for each token, allowing large model capacity with relatively limited computation. However, the large size and irregular access pattern of expert weights can make **expert-data movement a major memory bottleneck** during inference. Recent work shows that expert accesses can exhibit predictable patterns that can be exploited to prefetch experts before they are needed. [2], [3]

In this project, you will first understand and reproduce the **Duplex** architecture [1] using the provided ***LLMSimulator***. Duplex combines conventional compute with Logic-PIM and dynamically selects the appropriate compute engine for different LLM operations.

After reproducing the baseline, you will modify the system to study a simplified **single-GPU MoE architecture**. Multi-GPU expert sharding and Logic-PIM will be disabled, and an additional **off-chip DRAM** will be modeled to store all expert weights. The GPU HBM will represent the fast local memory available to the computation. An A100-class 80 GB HBM configuration may be used as the reference local-memory configuration.

You will analyze the MoE execution flow and measure the time required to load experts from off-chip DRAM and to execute them on the GPU. Investigate whether expert loading is substantially more expensive than expert computation and explain the observed behavior using the MoE architecture, expert size, memory bandwidth, and data movement. Next, analyze the **temporal and spatial correlation** of expert accesses. Use appropriate statistics or visualizations, such as expert-frequency distributions, heatmaps, transition probabilities, or correlation matrices, to determine whether future expert requests can be predicted from previous requests. ST-MoE, for example, exploits correlations across adjacent MoE layers and consecutive decoding tokens to predict future expert activations. [2]

Based on this analysis, design a **training-free expert predictor** using a suitable technique inspired by ST-MoE, APEX, or another method. APEX similarly exploits early prediction and dynamically determines how many additional experts to prefetch, rather than using a fixed prefetch budget. [3] The predictor must be evaluated against the actual routed experts using an appropriate prediction/overlap metric.

Finally, use the predictor to design and implement an **expert prefetcher** that transfers predicted experts from off-chip DRAM into GPU HBM before they are required. The objective is to overlap expert loading with useful computation and reduce the number of exposed memory stalls. Compare the proposed design with the original on-demand execution in terms of *latency, expert-loading stalls, memory traffic, and energy consumption*.

Deliverable:

Mid Evaluation:

1. Reproduce the provided Duplex configuration and demonstrate a working simulation.
2. Disable multi-GPU expert sharding and Logic-PIM, add off-chip DRAM, and place all expert weights in off-chip memory. Report expert loading time and expert execution time.
3. Explain the MoE execution flow and analyze why expert loading becomes a performance bottleneck.
4. Analyze the temporal/spatial correlation in expert access and present suitable plots/statistics.

Final Deliverable:

1. Implement a training-free expert prediction technique and evaluate prediction accuracy/overlap against the actual routed experts.
2. Design and implement a prefetching mechanism using the predictor to fetch future experts into the GPU HBM.
3. Compare baseline and prefetching configurations in terms of expert-load latency, total/token latency, exposed stall time, and other relevant metrics.
4. Evaluate energy consumption and energy savings due to reduced exposed memory stalls and discuss the additional memory traffic caused by prefetching.
5. The final report and complete codebase. Only one submission is required per group.

Note: For this project, all experiments **must be performed using simulation; students are not required to have access to a physical GPU.** The primary simulator is **LLMSimulator**, based on the Duplex architecture, with its memory-system modeling using a modified Ramulator 2.0. The **SSD-offloading framework** may be used for operation-level latency and energy modeling of LLM inference with MoE expert weights stored in CPU memory or SSD. Before beginning implementation, read the referenced papers and understand the architectural motivation before. The use of AI tools is permitted for coding and debugging, but the report must be written by the students themselves.

Tools Used:
LLMSimulator: https://github.com/scale-snu/LLMSimulator
SSD-offloading: https://github.com/scale-snu/SSD-offloading

The SSD-offloading framework builds on LLMSimulator and provides latency and energy breakdowns for LLM inference, including MoE expert weights offloaded to CPU memory or SSD.

Outcome: After completion, you will gain hands-on experience in computer architecture for AI/LLM systems, including MoE workload characterization, memory-hierarchy analysis, expert data movement, locality, prefetching, latency hiding, and performance-energy trade-offs.

REFERENCES:
[1] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, “Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batching,” 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024, pp. 1429–1443, doi: 10.1109/MICRO61859.2024.00105.

[2] Y. Zhao, R. C. Bunescu, A. Louri, A. Karanth, and K. Wang, “A Spatio-Temporal Expert Prefetching Framework for Efficient MoE-based LLM Inference,” arXiv:2606.15453, 2026.

[3] A. Kanani, L. Badawi, and U. Y. Ogras, “APEX: Adaptive Expert Prefetching for Memory-Efficient Edge MoE Inference,” arXiv:2608.11688, 2026; accepted at IEEE/ACM CODES 2026.

TA contact:
Ayesha Parveen
Email: ayeshap@iiitd.ac.in

Project Title: Characterizing Memory Access Predictability Using Hardware Prefetcher Classes and improving the prefetcher.

Objective: Different hardware prefetchers exploit different forms of memory-access predictability/patterns. This project studies which prefetcher class is effective for which workload, how much of the workload's cache-miss stream each class can cover, and why each prefetcher succeeds or fails. Students will evaluate three representative prefetcher classes on a fixed set of workloads, relate the measured prefetch coverage to properties of the underlying memory-access stream, and explore feedback-directed techniques to improve the effectiveness of the IP-stride prefetcher.

The project should answer:

1. Which workloads exhibit strong PC-local stride predictability?
2. Which workloads benefit from ordered delta-history/correlation?
3. Which workloads exhibit strong page-local spatial predictability?
4. Which workloads are not well captured by any of these classes?
5. When high coverage does not translate into performance, what prevents the benefit?

Tools/Simulator: Champsim

System Configuration:

Core	1 core, 4-Ghz, 8-wide Fetch/Decode/Dispatch/Execute/Retire, 576-entry ROB, 240-entry LQ, 112-entry SQ
L1D	48 KB, 12-way, 5-cycle latency, 16-entry MSHRs
L2C	2 MB, 16-way, 10-cycle latency, 32-entry MSHRs
LLC	3 MB, 12-way, 35-cycle latency, 64-entry MSHRs
DRAM	4800 MT/s, tCAS: 36, tRCD: 36, tRP: 36, tRAS: 78,

Prefetchers:

All prefetchers are evaluated only at the L2 cache. L1D and LLC prefetching must remain disabled. Four configurations are required:

Configuration	L2 Prefetcher	Predictor Class
No pref	None	Baseline
Stride	ip_stride	PC-local stride
SPP	spp_dev	Delta-history/correlation
AMPM	va_ampm_lite	Spatial/access-map

Workloads:

SPEC2017- 603.bwaves_s-891B, 619.lbm_s-4268B, 649.fotonik3d_s-1176B, 605.mcf_s-1554B, 620.omnetpp_s-141B, 623.xalancbmk_s-165B

Graph- Ligra BFS/Connected-Components trace, Ligra PageRankDelta trace, Ligra Triangle trace

AI/ML- Llama2 trace

For single-core configurations, each must be warmed up for 50 million (M) instructions and evaluated for the next 100M instructions. The traces can be downloaded from here: https://drive.google.com/drive/folders/1KiRZHRw811E1-zjlWy3AaBfZQnW4kQA_ .

Metrics to be computed:-

1. Baseline L2C MPKI= $\frac{L2\ demand\ misses}{instructions/1000}$
2. Coverage=$\frac{Misses\ in\ no\ prefetch - Misses\ with\ prefetch}{Misses\ in\ no\ prefetch}$
3. Accuracy= $\frac{Useful\ Prefetches}{Prefetch\ fill}$
4. IPC speedup
5. Memory traffic

Memory Access Characterization:

Simulation results alone are insufficient. Students must explain why a prefetcher obtains or fails to obtain coverage. The supplied baseline configuration will dump a bounded sample of L2 demand accesses containing:

a. Load PC
b. Cache-line address
c. Access order
d. L2 hit/miss information.

A student must analyze the following three properties:

A. PC-local Delta Behavior
   1) For important load PCs, determine:
   2) number of L2 misses,
   3) most frequent address delta,
   4) fraction of accesses represented by that delta.

   A high dominant-delta fraction indicates stride/local regularity.

B. Ordered Delta History

   Analyze the L2 address stream for recurring sequences such as: +1, +3, +1, +3, ..

   Repeated ordered delta histories provide evidence for correlation-based predictability.

C. Spatial Footprints

   Divide each 4-KB page into 64 cache lines and examine which offsets are repeatedly accessed. Similar footprints across different pages, even when access order changes, indicate spatial predictability.

Mid-evaluation Deliverable- Mechanism Understanding and Characterization

1. Using the traces and running for all the 4 configurations.
2. Report of the metrics mentioned in the description.
3. Memory Access Characterization.
4. Explanation of the three prefetchers

End-evaluation.

1. What is the root cause of failure in each case?
2. For each, show evidence from the access stream and answer:
   a. What pattern does the winning prefetcher exploit?
   b. Why do the other predictors fail?
   c. Does high coverage translate into IPC improvement?
   d. If not, is the likely cause accuracy, timeliness, or additional traffic?
3. Using the FDP [1] concept improves the performance of the IP-stride prefetcher. You can use ideas from any recent SOTA paper, such as [2], to improve performance.

**Students must clearly motivate their chosen feedback signal and control mechanism, describe the hardware changes, and evaluate whether the modification improves performance and/or bandwidth efficiency over the original IP-stride prefetcher. The modification should retain the fundamental IP-stride predictor; replacing it with another prefetcher is not permitted.

Outcome: After completion, students will gain hands-on experience in hardware data prefetching, memory-access pattern characterization, cache-hierarchy behavior, prefetch coverage and accuracy analysis, timeliness, memory-traffic trade-offs, and feedback-directed prefetcher optimization using ChampSim.

Reference:

1. S. Srinath, O. Mutlu, H. Kim and Y. N. Patt, "Feedback Directed Prefetching: Improving the Performance and Bandwidth-Efficiency of Hardware Prefetchers," 2007 IEEE 13th International Symposium on High Performance Computer Architecture, Scottsdale, AZ, USA, 2007, pp. 63-74, doi: 10.1109/HPCA.2007.346185
2. Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. DSPatch: Dual Spatial Pattern Prefetcher. In Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52). Association for Computing Machinery, New York, NY, USA, 531–544. https://doi.org/10.1145/3352460.3358325

TA Contact

Naorem Akshaykumar
Email: naorema@iiitd.ac.in

Project 4 : RTL Design & Verification of MESI Cache Coherence Protocol with Next-Line Prefetching

Description :

In this project, you will design, implement, and verify an RTL model of a **multicore cache subsystem implementing the MESI cache-coherence protocol along with a Next-Line Prefetcher**.

The proposed system will consist of at least **two private L1 caches, a shared L2 cache, and main memory**. There is no requirement to implement an actual CPU; processor read/write requests can be generated through the testbench. The private L1 caches will maintain coherence using the **MESI (Modified, Exclusive, Shared, Invalid) protocol** over a shared snooping bus/interconnect.

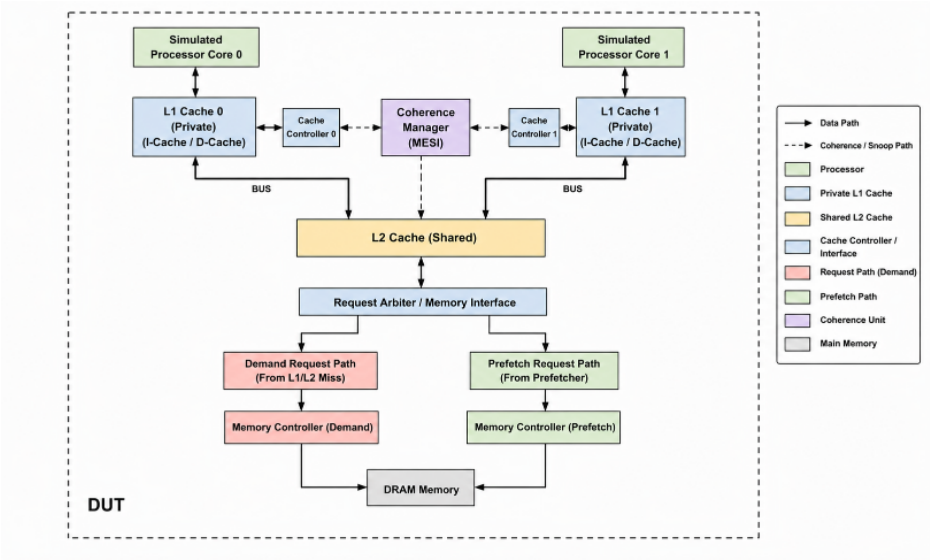
Students will implement correct MESI state transitions and snoop handling for operations such as read requests, write requests, invalidations, upgrades, interventions, and write-backs.

After implementing the coherent cache hierarchy, students will integrate a **Next-Line Prefetcher**. For an accessed cache line, the prefetcher predicts that the following cache line may be required and fetches it in advance. The objective is to study how a simple prefetching mechanism affects **cache miss rate, memory-access latency, memory traffic, and overall performance**.

Students are free to choose cache and memory parameters such as **L1/L2 size, main-memory size, cache-line size, associativity, replacement policy, and access latency**, but all design choices must be stated and justified.

The following latency constraint must be maintained:

L1 access latency < L2 access latency < Main-memory access latency



Mid-Evaluation Deliverables

1. Short resource list containing papers/videos/material used.
2. Proposed system architecture and block diagram.
3. Design choices:
    - L1/L2 cache size
    - Main-memory size
    - Cache-line size
    - Associativity
    - Replacement policy
    - L1, L2, and memory latency
4. Justification of selected architectural parameters.
5. MESI state-transition/FSM diagram.
6. Test plan containing directed read/write and sharing scenarios.
7. Project schedule and team roles.
8. RTL implementation of the **L1, L2, and main-memory subsystem with basic read/write operations**.
9. Demonstration of cache hits, misses, L2 accesses, and main-memory accesses with the chosen latency values.

End-Evaluation Deliverables

1. Implement the **MESI cache coherence protocol** using Verilog/SystemVerilog.
2. Implement and simulate **all possible test cases and conflict scenarios** to verify correct read/write coherence and MESI state transitions between the caches, bus/interconnect, and memory.
3. Implement and integrate a **Next-Line Prefetcher** with the cache subsystem.
4. Compare the system **with and without prefetching** in terms of cache performance, access latency, memory traffic, and overall performance.

Tools & Recommended Simulators

- Verilog/SystemVerilog
- Icarus Verilog + GTKWave or
- Vivado Simulator

Functional correctness through simulation (waveform) is the primary requirement.

References

[1] K. Loughlin, S. Saroiu, A. Wolman, Y. A. Manerkar, and B. Kasikci, “MOESI-prime: Preventing Coherence-Induced Hammering in Commodity Workloads,” in Proceedings of the 49th International Symposium on Computer Architecture (ISCA), 2022. Available: https://www.microsoft.com/en-us/research/publication/moesi-prime-preventing-coherence-induced-hammering-in-commodity-workloads/

[2] B. Panda, “CLIP: Load Criticality Based Data Prefetching for Bandwidth-Constrained Many-Core Systems,” in Proceedings of the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO), pp. 714–727, 2023. Available: https://ieeexplore.ieee.org/search/searchresult.jsp?newsearch=true&queryText=CLIP%20Load%20

Criticality%20Based%20Data%20Prefetching

[3] IEEE Xplore, “IEEE Document 10764627,” 2024. Available: https://ieeexplore.ieee.org/document/10764627

[4] J. Fang, Y. Xu, H. Kong, and M. Cai, “A Prefetch Control Strategy Based on Improved Hill-Climbing Method in Asymmetric Multi-Core Architecture,” The Journal of Supercomputing, vol. 79, pp. 10570–10588, 2023. Available: https://link.springer.com/article/10.1007/s11227-023-05078-6

[5] H. K. Kapoor, “Cache Coherence / MESI Protocol,” IIT Guwahati, Video Lecture, YouTube. Available: https://www.youtube.com/watch?v=bBSmcFoFRYw

[6] “Prefetching and Prefetcher Design,” Video Lecture, YouTube. Available: https://www.youtube.com/live/oh8JhsCKxXk

Project 5 - Performance Characterization and Optimization of MoE Communication on Chiplet-Based Architectures

Project Description: Chiplet-based architectures are emerging as an effective approach for building large and scalable computing systems. Instead of integrating all processing components on a single large die, they use multiple smaller dies, or chiplets, connected through an inter-chiplet network. Similarly, large-scale AI models such as Mixture-of-Experts (MoE) consist of multiple processing blocks called experts but activate only a small subset of these experts for each input token. When the experts are distributed across different chiplets, accessing an expert located on a remote chiplet requires communication across the inter-chiplet network. Since MoE workloads can exhibit highly uneven expert utilization, some experts may receive significantly more requests than others. This uneven communication can concentrate traffic on particular chiplets and communication links, creating hotspots that increase packet latency, congestion, and overall communication overhead.

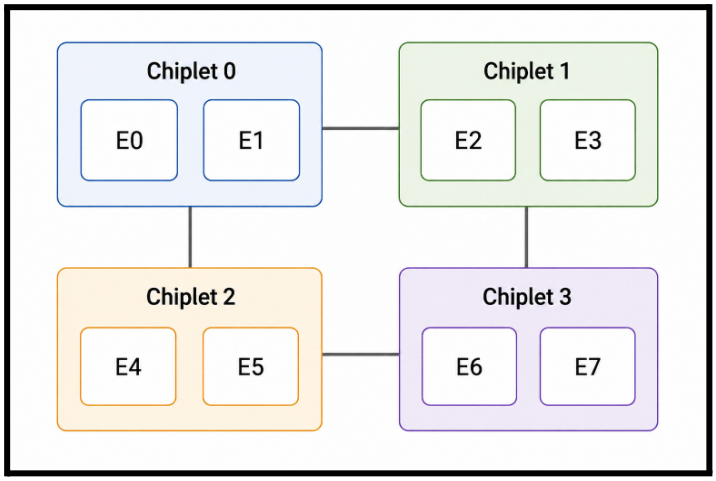


Fig : 4-Chiplet Architecture with Distributed MoE Experts

The objective of this project is to understand how uneven expert selection in Mixture-of-Experts (MoE) models affects communication in chiplet-based systems. Specifically, we will determine how expert-selection imbalance, network topology, and expert placement contribute to communication hotspots, increased congestion, and higher packet latency, and identify when these hotspots become a significant performance bottleneck. To study this, we will use **RapidChiplet** as the main simulation framework, with its integrated **BookSim2** backend for detailed cycle-level network simulation. A framework will be developed to generate configurable MoE expert-selection patterns with different levels of expert-selection skew, map experts to different chiplets, and translate the resulting expert requests into chiplet-to-chiplet communication traces. Using this simulation flow, we will study how factors such as the number of chiplets, network topology, routing strategy, link

latency, communication load, and expert placement influence network performance under both balanced and imbalanced MoE workloads. The communication behavior will be evaluated using metrics such as average packet latency, network throughput, average hop count, link utilization, maximum link utilization, and remote communication ratio. Link-utilization heatmaps will help identify chiplets and communication links that carry unusually high traffic. Based on these results, we will investigate improved ways to place experts across chiplets and selectively replicate frequently accessed experts to reduce traffic concentration and network congestion. These methods will be compared with the baseline expert placement to determine whether changing expert placement is sufficient, when replication provides additional benefits, and what extra storage and communication costs it introduces.

Mid-Evaluation Deliverables

1. Set up RapidChiplet and BookSim2 and verify that the simulator works using an existing example.
2. Create a basic 4-chiplet or 8-chiplet system with a fixed network topology, routing method, and link settings.
3. Develop an MoE workload generator that can create balanced and imbalanced expert-selection patterns, and map the experts to different chiplets.
4. Convert the generated MoE expert requests into chiplet-to-chiplet communication traces that can be used by RapidChiplet.
5. Run simulations for balanced, moderately imbalanced, and hot-expert workloads and measure packet latency, throughput, hop count, link utilization, maximum link utilization, and remote communication ratio.
6. Generate link-utilization heatmaps and analyze how uneven expert selection creates communication hotspots and affects network performance.

End-Evaluation Deliverables

1. Complete the full simulation flow from MoE workload generation to performance analysis using RapidChiplet and BookSim2.
2. Test different MoE workload conditions, including balanced expert usage, one hot expert, multiple hot experts, and changing hot experts over time.
3. Study how network performance changes by varying parameters such as the number of chiplets, network topology, link latency, traffic load, expert placement, and routing strategy.
4. Identify the main communication bottlenecks, such as hotspot chiplets, heavily used links, high packet latency, and network congestion.

5. Develop and test better expert-placement strategies and selectively replicate frequently used experts to reduce network congestion.
6. Compare the baseline and optimized architectures using packet latency, throughput, hop count, link utilization, maximum link utilization, and remote communication ratio.
7. Analyze why the bottlenecks occur, how improved expert placement helps, when expert replication is useful, and what extra storage or communication cost it introduces.
8. Identify the workload and architecture conditions under which the proposed optimizations provide the best performance improvement.

References

[1] P. Iff, B. Bruggmann, B. Morel, M. Besta, L. Benini, and T. Hoefler, “RapidChiplet: A Toolchain for Rapid Design Space Exploration of Inter-Chiplet Interconnects,” in *Proceedings of the 22nd ACM International Conference on Computing Frontiers (CF)*, 2025, pp. 168–171. doi: 10.1145/3719276.3725170.

[2] N. Jiang, J. Balfour, D. U. Becker, B. Towles, W. J. Dally, G. Michelogiannakis, and J. Kim, “A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator,” in *2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2013, pp. 86–96. doi: 10.1109/ISPASS.2013.6557149.

[3] X. Tang et al., MoEntwine: Unleashing the Potential of Wafer-scale Chips for Large-scale Expert Parallel Inference, in 2026 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2026. doi: 10.1109/HPCA68181.2026.11408594.

[4] H. Kang, Z. Yu, and C. Xiong, “FLAME-MoE: A Transparent End-to-End Research Platform for Mixture-of-Experts Language Models,” arXiv:2505.20225, 2025.

Tools : RapidChiplet- https://github.com/spcl/rapidchiplet
Booksim 2.0 - https://github.com/booksim/booksim2

TA Contact: Sameena khan

Email: sameenak@iiitd.ac.in